



# URL SHORTENER SYSTEM DESIGN

By Abhinave P.B | September 2026

```
PS C:\Users\abhin\OneDrive\Desktop\Ironfleet\Archives\URL Shortener> curl -I http://localhost:5000/1
HTTP/1.1 302 FOUND
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Wed, 30 Sep 2026 10:29:23 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 231
Location: https://www.google.com
Connection: close

PS C:\Users\abhin\OneDrive\Desktop\Ironfleet\Archives\URL Shortener> docker compose logs web --tail 5
web-1 | Press CTRL+C to quit
web-1 | 172.21.0.1 - - [30/Sep/2026 10:27:23] "POST /shorten HTTP/1.1" 201 -
web-1 | 172.21.0.1 - - [30/Sep/2026 10:28:19] "POST /shorten HTTP/1.1" 201 -
web-1 | CACHE HIT
web-1 | 172.21.0.1 - - [30/Sep/2026 10:29:23] "HEAD /1 HTTP/1.1" 302 -
PS C:\Users\abhin\OneDrive\Desktop\Ironfleet\Archives\URL Shortener>
```

## Project Overview

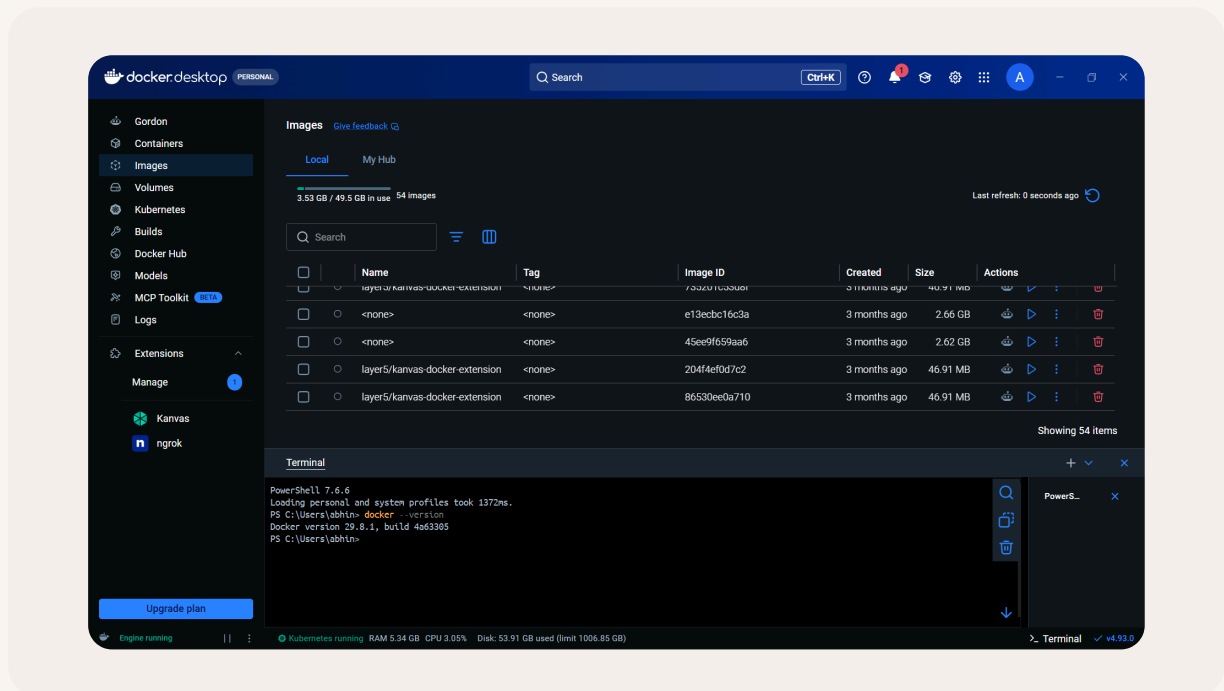
### What we're building and why

In this project, I'm building a scalable URL shortener using Flask, PostgreSQL, Redis, and Docker so that I can learn how backend services, databases, caching, and containerized applications work together in a real-world system.

# Setting Up the Environment

## Environment setup goals

In this step, I'm setting up my development environment with a terminal, Docker Desktop, and Cursor so that I can build, run, and manage the different services of my URL shortener project.



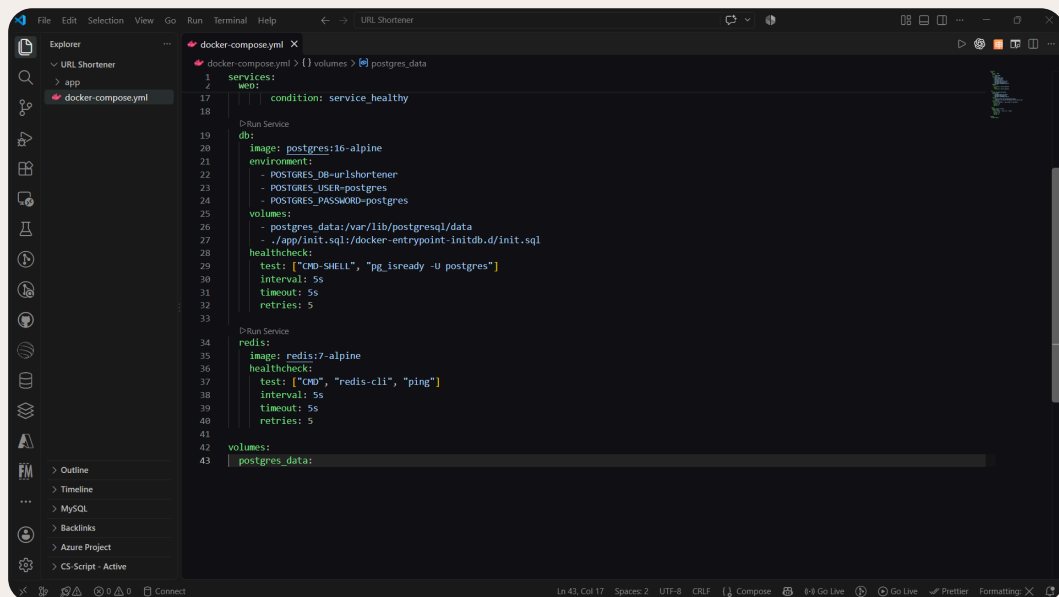
## The three core services

The three services are Flask API, PostgreSQL, and Redis. The first handles API requests and URL redirection. The second stores the URL mappings and application data. The third provides fast caching for frequently accessed URLs.

# Designing the System Architecture

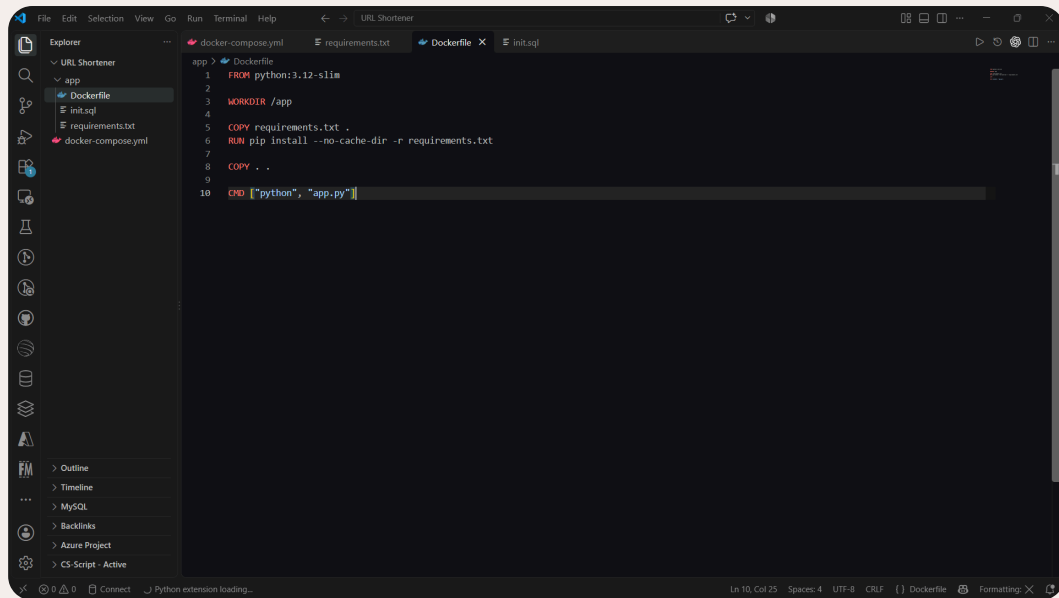
## Architecture design goals

In this step, I'm creating the Docker Compose configuration, container setup, Python dependencies, and PostgreSQL database schema so that I can define and run the three-service URL shortener architecture consistently.



## Services defined in docker-compose.yml

The three services are web, db, and Redis. Each one handles a specific part of the URL shortener: web handles API requests and redirects, db stores URL mappings permanently in PostgreSQL, and Redis provides fast caching for frequently accessed URLs.



## Infrastructure file deep dive

I chose PostgreSQL as the database. Its role is to store the URL mappings permanently, including the unique ID, original long URL, and creation timestamp, so the data remains available even when the application restarts.

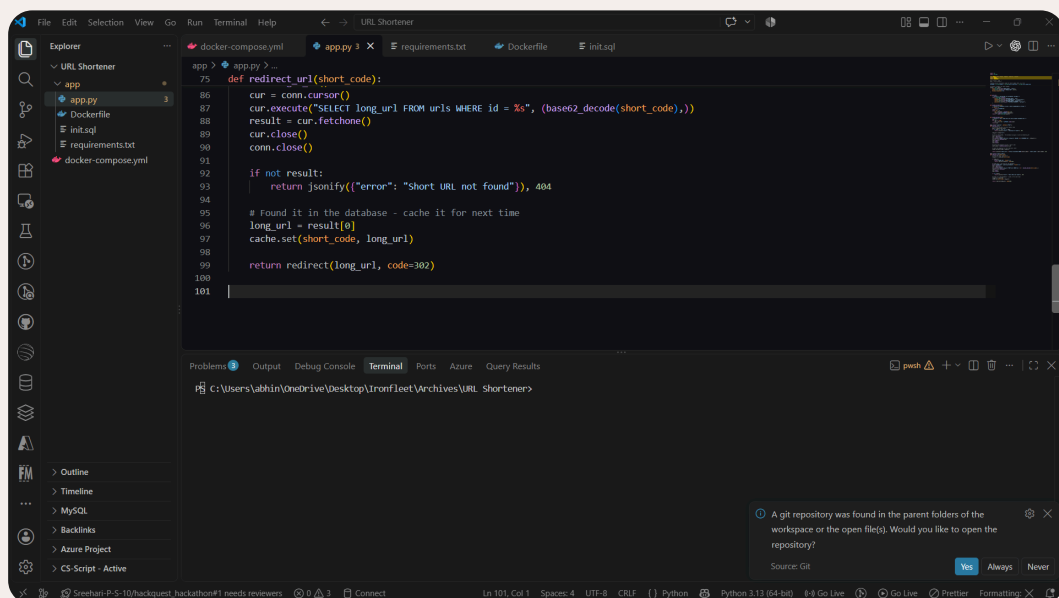
# Building the URL Shortener Logic

## Application logic goals

In this step, I'm writing the Flask API logic in `app.py` so that I can create short URLs from long URLs, redirect users to the original URLs, and connect the API with PostgreSQL and Redis.

## The URL shortening flow

When a long URL hits /shorten, the app first stores the URL in PostgreSQL and gets a unique database ID, then it converts that ID into a Base62 short code and caches the mapping in Redis, and finally returns the generated short URL and short code to the user.



```
def redirect_url(short_code):
    cur = conn.cursor()
    cur.execute("SELECT long_url FROM urls WHERE id = %s", (base62.decode(short_code),))
    result = cur.fetchone()
    cur.close()
    conn.close()

    if not result:
        return jsonify({"error": "Short URL not found"}), 404

    # Found it in the database - cache it for next time
    long_url = result[0]
    cache.set(short_code, long_url)

    return redirect(long_url, code=302)
```

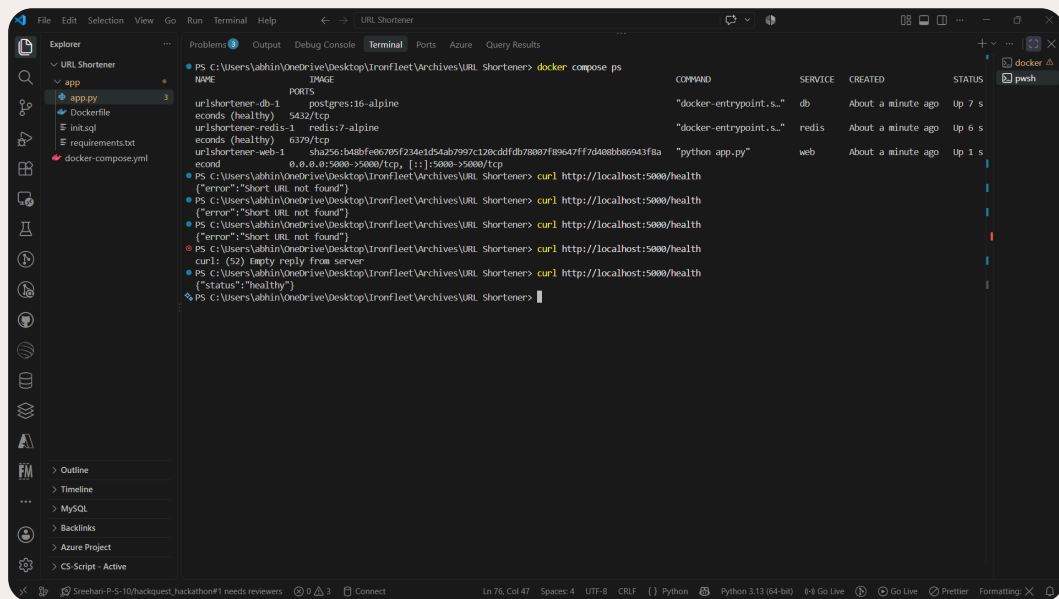
## Handling redirect lookups

When someone visits a short URL, the app first checks Redis for the short code to quickly find the original URL. If that misses, it decodes the short code into the database ID, queries PostgreSQL for the original URL, and then redirects the user to it.

# Launching and Verifying the System

## Launch goals

In this step, I'm starting the Flask API, PostgreSQL database, and Redis cache using Docker Compose so that I can run all three services together and verify that they are healthy and communicating correctly.



The screenshot shows the Visual Studio Code interface with the Docker Compose services running. The Explorer pane on the left shows the project structure with files like `app.py`, `Dockerfile`, `init.sql`, `requirements.txt`, and `docker-compose.yml`. The Docker container list on the right shows three services: `urlshortener-db-1` (PostgreSQL), `urlshortener-redis-1` (Redis), and `urlshortener-web-1` (Flask API). The terminal window shows the output of the `docker compose ps` command and several `curl` commands testing the `http://localhost:5000/health` endpoint. The output of the `curl` commands shows the response `{\"error\": \"Short URL not found\"}`.

| NAME                 | IMAGE              | COMMAND                  | SERVICE | CREATED            | STATUS |
|----------------------|--------------------|--------------------------|---------|--------------------|--------|
| urlshortener-db-1    | postgres:16-alpine | "docker-entrypoint.s..." | db      | About a minute ago | Up 7 s |
| urlshortener-redis-1 | redis:7-alpine     | "docker-entrypoint.s..." | redis   | About a minute ago | Up 6 s |
| urlshortener-web-1   | python:3.13-alpine | "python app.py"          | web     | About a minute ago | Up 1 s |

```
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
urlshortener-db-1                   postgres:16-alpine                 "docker-entrypoint.s..."           db         About a minute ago    Up 7 s
urlshortener-redis-1               redis:7-alpine                     "docker-entrypoint.s..."           redis      About a minute ago    Up 6 s
urlshortener-web-1                 python:3.13-alpine                 "python app.py"                      web        About a minute ago    Up 1 s

PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> curl http://localhost:5000/health
{"error": "Short URL not found"}
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> curl http://localhost:5000/health
{"error": "Short URL not found"}
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> curl http://localhost:5000/health
{"error": "Short URL not found"}
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> curl http://localhost:5000/health
curl: (52) empty reply from server
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> curl http://localhost:5000/health
{"status": "healthy"}
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener>
```

## Health endpoint response

When I ran `curl http://localhost:5000/health`, I saw:

```
{"error": "Short URL not found"}
```

This happened because the `/health` endpoint was not defined, so Flask treated `health` as a `short_code` using the `/<short_code>` route.

# Testing and Observing the System

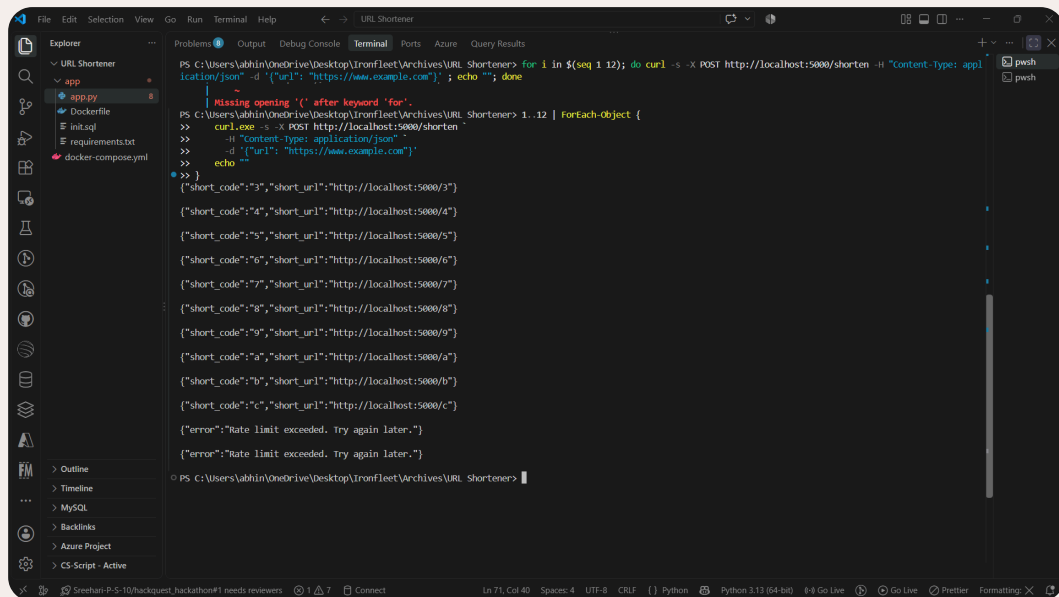
## Testing goals

In this step, I'm testing URL creation, redirection, and cache behavior so that I can observe how Base62 short codes are generated, how redirects work with HTTP status codes, and how the cache-aside pattern retrieves data from Redis before falling back to PostgreSQL.

## Redis cache flush behavior

After flushing Redis, the app had to read the original link from database instead of the Redis cache

# Implementing Rate Limiting with Redis



```
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> for $i in $(seq 1 12); do curl -s -X POST http://localhost:5000/shorten -H "Content-Type: application/json" -d '{"url": "https://www.example.com"}'; echo ""; done
Missing opening '(' after keyword 'for'.
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener> 1..12 | ForEach-Object {
>> curl.exe -s -X POST http://localhost:5000/shorten
>> -H "Content-Type: application/json"
>> -d '{"url": "https://www.example.com"}'
>> echo ""
{"short_code":"3","short_url":"http://localhost:5000/3"}
{"short_code":"4","short_url":"http://localhost:5000/4"}
{"short_code":"5","short_url":"http://localhost:5000/5"}
{"short_code":"6","short_url":"http://localhost:5000/6"}
{"short_code":"7","short_url":"http://localhost:5000/7"}
{"short_code":"8","short_url":"http://localhost:5000/8"}
{"short_code":"9","short_url":"http://localhost:5000/9"}
{"short_code":"a","short_url":"http://localhost:5000/a"}
{"short_code":"b","short_url":"http://localhost:5000/b"}
{"short_code":"c","short_url":"http://localhost:5000/c"}
{"error":"Rate limit exceeded. Try again later."}
{"error":"Rate limit exceeded. Try again later."}
PS C:\Users\abhin\OneDrive\Desktop\IronFleet\Archives\URL Shortener>
```

## Why Redis excels at rate limiting

In this project extension, Redis is better for rate limiting because it is an in-memory store with very fast reads/writes, supports atomic counter operations, and can automatically expire keys using TTL. This makes it ideal for tracking request counts over short time windows without putting extra load on PostgreSQL.



# Reflections and Key Takeaways

## Tools and concepts learned

The key tools I used include Python Flask, PostgreSQL, Redis, Docker, and Docker Compose. Key concepts I learnt include REST APIs, Base62 encoding, database persistence, caching, cache-aside architecture, HTTP redirects, health checks, and multi-service container orchestration.

## Time and challenges

This project took me approximately one hour. The most challenging part was running and verifying.

## Final thoughts

I did this project today to learn how to build and run a multi-service backend application using Flask, PostgreSQL, Redis, and Docker. Another skill I want to learn is deploying and scaling backend applications in the cloud.